Proyecto final de Base de Datos – Parte 2

Arquitectura de datos para Ciencia de Datos e IA:
Predicción y detección de anomalías de demanda espaciotemporal de viajes (NYC TLC HVFHV 2026)

Sebastian Rodrigo García Villacorta
Dante Guillermo Barreto Daza
Clinton Michael Chulluncuy Reynoso
Ronald Ramiro Hilario Orihuela

Curso: Base de Datos (MCD8008) – Docente: Luis Alberto Chicoma Chevez

Resumen

En la primera entrega se implementó un modelo relacional en PostgreSQL sobre los registros High Volume For-Hire Vehicle (HVFHV) de Nueva York (enero–mayo de 2026), con 11 tablas, particionado mensual, control de calidad y feature engineering orientado a la predicción de demanda. En esta segunda etapa se amplía el caso hacia una arquitectura especializada complementaria: se selecciona **TimescaleDB** (series temporales) junto con **PostGIS** (espacial) para responder en tiempo casi real a la pregunta *¿cuántos viajes se solicitarán en la próxima hora por zona y plataforma, y qué zonas presentan demanda anómala?*. Se justifica la elección por el patrón de acceso temporal y espacial de los datos, se diseña el modelo de datos especializado y se presentan resultados reales de la implementación, incluyendo una reducción de almacenamiento de 162 MB a 14 MB mediante compresión nativa (factor $\approx 11{,}6\times$).

Índice

1. Arquitectura de datos para Ciencia de Datos e IA **2**

2. Definición del caso de IA / Ciencia de Datos **2**
2.1. Enunciado . . . 2
2.2. Unidad predictiva y variable objetivo . . . 2
2.3. Salidas esperadas . . . 2

3. Caracterización del problema **2**

4. Selección de tecnología **2**

5. Modelo de datos NoSQL / especializado **3**

6. Arquitectura propuesta **3**

7. Comparación entre tecnologías **4**
7.1. ¿Por qué utilizar ambas tecnologías en lugar de reemplazar una por otra? . . . 4

8. Resultados de la implementación **5**
8.1. Caracterización . . . 5
8.2. Consultas temporales . . . 5
8.3. Consultas espaciales . . . 5

9. Pasos de replicación **5**

1. Arquitectura de datos para Ciencia de Datos e IA

El caso desarrollado en la Parte 1 modela los viajes de movilidad bajo demanda de Uber y Lyft publicados por la *New York City Taxi and Limousine Commission*. El proceso observable comprende solicitud, asignación, llegada, recogida, desplazamiento, finalización y componentes económicos de cada viaje.

Las características del problema — volumen elevado, generación periódica de datos, análisis predominantemente por rangos de tiempo y por zona geográfica — hacen que el modelo relacional sea insuficiente para consultas analíticas de baja latencia sobre series históricas largas. Por ello se identifica un escenario de Ciencia de Datos que puede beneficiarse de una tecnología de almacenamiento y procesamiento especializada, **sin reemplazar** a PostgreSQL, que sigue siendo la fuente de verdad transaccional.

2. Definición del caso de IA / Ciencia de Datos

2.1. Enunciado

Caso: predicción de demanda espaciotemporal de viajes y detección de anomalías.

¿Cuántos viajes se van a solicitar en la próxima hora en cada zona y plataforma, y qué zonas presentan una demanda inusualmente alta o baja respecto de su comportamiento habitual?

2.2. Unidad predictiva y variable objetivo

La unidad predictiva, heredada de la Parte 1, es la combinación

$$(\text{hora},\ \text{zona_origen_id},\ \text{plataforma_codigo}),$$

y la variable objetivo es viajes_objetivo, es decir, el número de recogidas observadas en esa hora, zona y plataforma.

2.3. Salidas esperadas

- Pronóstico horario de demanda por zona y plataforma (tarea de regresión).
- Alerta de anomalía cuando la demanda observada se aleja del comportamiento esperado.

3. Caracterización del problema

4. Selección de tecnología

Se selecciona **TimescaleDB** (extensión de PostgreSQL para series temporales) como tecnología principal, complementada con **PostGIS** para capacidades espaciales. La selección se deriva directamente de la caracterización anterior:

- **Qué problema resuelve:** consultas de rango temporal y agregaciones sobre millones de observaciones con baja latencia, que en un escaneo secuencial del modelo relacional son costosas.
- **Por qué fue seleccionada:** el dato es una serie temporal; TimescaleDB aporta hypertables (particionado automático por tiempo), compresión nativa y *continuous aggregates*, y al ser una extensión se integra con el modelo relacional existente sin migración.
- **Ventajas:** compresión demostrada (162 MB $\rightarrow$ 14 MB), agregados mantenidos automáticamente y consultas espaciales vía PostGIS/GiST.

Dimensión	Valor / estimación
Volumen	105 996 114 filas en el universo (ene–may 2026); muestra reproducible de 1 000 000 filas/mes (4 999 931 cargadas). Proyección $\approx$ 250 M filas/año.
Frecuencia	Ingesta mensual por lotes; análisis horario (agregado `demanda_hora`).
Tipos de datos	Temporales (timestamp), numéricos (importes, millas, duración), booleanos y geoespaciales (longitud/latitud WGS84).
Tiempo real	Casi real: inferencia cada hora sobre la serie actualizada.
Latencia esperada	Inferencia $<$ 100 ms; consulta de demanda $<$ 500 ms.
Patrón lect/escritura	Escritura en lote (ingesta mensual) y lectura analítica intensiva por rango temporal y por zona.
Escalabilidad	Particionado por tiempo + compresión para sostener $\approx$ 250 M filas/año.
Consultas predominantes	Agregados temporales (`time_bucket`), rezagos, medias móviles y filtros por zona/proximidad.

Cuadro 1: Caracterización del problema de predicción de demanda.

- **Limitaciones:** la compresión de segmentos deshabilita algunas escrituras directas sobre chunks comprimidos (requiere descompresión/recompresión) y el paralelismo de escritura es menor que en soluciones distribuidas puras.
- **Cómo se complementa:** PostgreSQL conserva el núcleo transaccional (viajes, catálogos, auditoría, integridad referencial); TimescaleDB/PostGIS cubren el patrón de acceso analítico temporal–espacial.

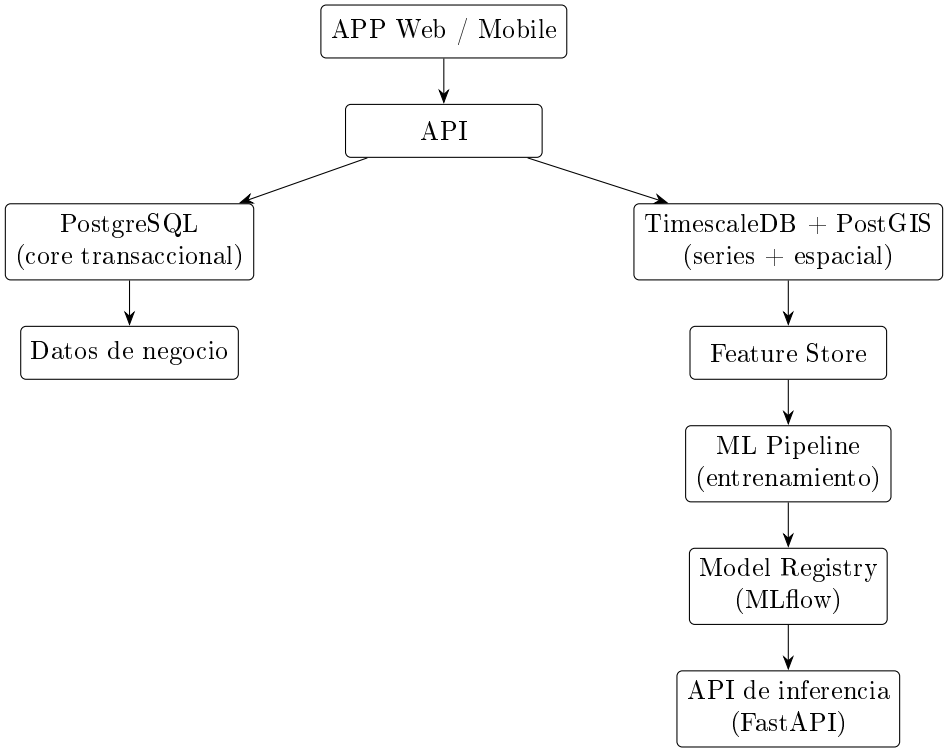
5. Modelo de datos NoSQL / especializado

El modelo especializado se describe en detalle en el entregable `modelo_nosql.pdf`. En síntesis:

1. Se añade una columna `geo GEOGRAPHY(POINT,4326)` a `core.zona_taxi`, derivada de longitud/latitud, con índice espacial GiST.
2. Se crea el hypertable `analytics.demanda_serie` particionado por `hora`.
3. Se crea un *continuous aggregate* diario `analytics.demanda_diaria`.
4. Se configura compresión por segmentos (`zona_origen_id`, `plataforma_codigo`) ordenados por `hora`.

6. Arquitectura propuesta

El diagrama completo se presenta en el entregable `arquitectura.pdf`. Los componentes principales son:



7. Comparación entre tecnologías

Criterio	**PostgreSQL**	**TimescaleDB (+PostGIS)**
Modelo	Relacional	Serie temporal + espacial
Particionado	Manual (por mes)	Automático (hypertable)
Compresión	No nativa	Nativa por segmentos ($\approx$ 11,6$\times$ observado)
Agregados históricos	Vistas/consultas costosas	Continuous aggregates
Consultas por rango temporal	Costosas sobre millones de filas	Optimizadas por tiempo
Espacial	No	PostGIS + índice GiST
Transacciones ACID	Excelente	Excelente (es PostgreSQL)
Caso principal	Core transaccional, integridad, catálogos	Analítica temporal–espacial en tiempo casi real

Cuadro 2: Comparación entre PostgreSQL y TimescaleDB/PostGIS.

7.1. ¿Por qué utilizar ambas tecnologías en lugar de reemplazar una por otra?

La respuesta se fundamenta en el concepto de *Polyglot Persistence*: distintas tecnologías coexisten según las características de cada problema. PostgreSQL permanece como la *fuente de verdad* transaccional (viajes, catálogos, lotes de ingesta, reglas de calidad e integridad referencial). TimescaleDB y PostGIS optimizan el patrón de acceso analítico temporal–espacial que el modelo relacional no resuelve eficientemente. Los datos fluyen de PostgreSQL hacia la capa especializada, sin duplicar la lógica de negocio ni sustituir el sistema de registro.

8. Resultados de la implementación

La implementación se ejecutó sobre PostgreSQL 16 + TimescaleDB 2.31 + PostGIS 3.6 en un contenedor Docker. Los resultados son reproducibles con los scripts de la carpeta `Sql/`.

8.1. Caracterización

- Viajes cargados: 4 999 931 (Uber 3 590 362 = 71,81 %; Lyft 1 409 569 = 28,19 %).
- Agregado horario relacional `demanda_hora`: 1 272 029 filas (162 MB).
- Hypertable `demanda_serie`: 1 272 029 filas, 22 chunks, todos comprimidos: 14 MB (factor $\approx 11{,}6\times$).

8.2. Consultas temporales

- Hora pico global: `2026-02-07 19:00` con 3 163 viajes.
- Día de mayor demanda: `2026-02-07` con 47 981 viajes.

8.3. Consultas espaciales

- Zonas con mayor demanda en un radio de 2 km de Midtown Center: Times Sq (55 043), Midtown Center (55 014), Midtown South (45 399).
- Zonas más cercanas a JFK (kNN): Springfield Gardens South (2 514 m), Springfield Gardens North (3 012 m), Baisley Park (3 570 m).
- Distancia de centroides Midtown–JFK: 20 324 m (12,63 millas), frente a 17,56 millas de media en los 1 040 viajes reales con esa ruta.

9. Pasos de replicación

1. Levantar el stack con `docker compose up -d -build` (carpeta `deploy/`).
2. Ejecutar en orden `01_schema.sql`, `02_tables.sql`, `03_data.sql` y `06_zonas_geo.sql`.
3. Cargar los CSV de `viaje` e `incidencia` con `\copy ... WITH (FORMAT csv, HEADER true, NULL '')`.
4. Reconstruir `analytics.demanda_hora`.
5. Ejecutar `07_timescale.sql` (capa espacial + hypertable + continuous aggregate + compresión).
6. Ejecutar `08_demo_queries.sql` para reproducir los resultados.